🚀 SQLcl + VS Code: Easy Guide for Database Object Export

🔷 What is SQLcl?

SQLcl (SQL Command Line) is the modern version of SQL*Plus.

- SQL*Plus → Old, simple, text-only
- SQLcl → Modern, smart, developer-friendly

✅ If you're a developer or APEX user, SQLcl is your best choice.

🔷 Step 1: Create Your Local Project Folder

Pick a folder where you will store your code and exports.
Example:

```
D:\eAgentVersionControl\SONALI_ABS
```

🔷 Step 2: Open SQLcl in VS Code

1. Right-click your database connection in VS Code
2. Click "Open SQLcl"
3. A terminal opens below
4. Change directory to your project folder:

🔷 Command

```
CD D:\eAgentVersionControl\SONALI_ABS
```

🔷 Step 3: Initialize SQLcl Project

Create a SQLcl project for your schema(s):

🔷 Command

```
project init -name SONALI_ABS -schemas EMOB
```

Multiple schemas:

```
project init -name SONALI_ABS -schemas EMOB,GUMS,BIOTPL,MEMIMG
```

🔷 Step 4: Export APEX Workspace / Applications

Export entire workspace:

```
apex export -expWorkspace -workspaceId 1400484024213076
```

Export all apps (Single File**):**

```
apex export -workspaceid 1400484024213076
```

Export all apps (Split):

```
apex export -workspaceid 1400484024213076 -split
```

List apps:

```
apex export -list -workspaceid 1400484024213076
```

Export single app:

```
apex export -applicationid 1234 -split

Export specific pages/LOVs:
apex export -applicationid 1234 -split -expComponents PAGE:3 PAGE:11
LOV:23618973754424510000
```

◈ Step 5: Export Database Objects

Export whole schema:

```
project export -schemas EMOB
```

Export single object:

```
project export -o FUNCTION_CALCULATE_SALARY
project export -o VIEW_EMP_INFO
```

◈ Step 6: Data Pump Export (Optional)

Create a directory:

```
CREATE OR REPLACE DIRECTORY EMOB_DUMP_DIR AS
'D:\eAgentVersionControl\CITY_ABS\DUMP';
```

Export schema:

```
dp export -dumpfile my_schema.dmp -logfile my_schema.log
```

Export multiple schemas:

```
dp export -schemas HR,DEMO -dumpfile multi_schema.dmp -dir
D:\eAgentVersionControl\CITY_ABS\DUMP
```

```
What is Our final Goal? Answer is Simple Source version controlling:
```

◈ Step 7: Git Integration

Initialize Git:

```
!git init
!git remote add origin
https://erainfotechbd@dev.azure.com/erainfotechbd/eAgent/_git/city_abs_
app
```

Pull latest code:

```
!git fetch origin
!git pull origin main --rebase
```

Create branch:

```
git checkout -b miaze_dev
```

```
git push --set-upstream origin miaze_dev
Set Git identity:
git config --global user.name "Ayub Miaze"
git config --global user.email ayub@erainfotechbd.com
```

◈ Recommended Daily Workflow

1. Pull latest changes:

```
git pull origin main --rebase
```

2. Export updated objects:

```
project export -schemas EMOB
```

3. Commit changes:

```
git add .
git commit -m "Updated EMOB schema objects"
```

```
4. Push to remote:
git push
```

◈ Pro Tips

- Use branches for each feature or bug fix (e.g., miaze_dev, feature_fix_salary)
- Use -split option for APEX apps (better version control)
- Write clear commit messages
- Regularly sync with git pull --rebase